

DP Computer Science: A 1.1.3 CPU and GPU Comparison (HL Only)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- 1. Explain the two design philosophies: why the CPU is built for serial, low-latency, varied work and the GPU for parallel throughput
- 2. Explain the reason behind each of the four comparison dimensions: core architecture, processing power, memory access, and power efficiency
- 3. Explain how the CPU and GPU divide work, share data over PCIe, and coordinate execution in a working system
- 4. Explain, from a researched real deployment, why the shape of the workload decides how work is split between the two processors

Key Vocabulary (Tier 3)

Term	Definition
Latency	The delay before a single operation produces its result
Throughput	The total amount of work completed per unit of time
Parallelism	Carrying out many operations at the same time
Core	An individual processing unit inside a CPU or GPU
Bandwidth	How much data can be moved to and from memory each second
Heterogeneous computing	A system that uses more than one type of processor together

Pedagogical Focus

- **Research Skills (ATL):** Students consciously practise research skills—each student or pair takes one real system (e.g., a named games console, cloud AI service, or weather-modelling cluster), locates a credible source, extracts how work is split between CPU and GPU, and records where the source came from.
- **Global Contexts (ATT):** The lesson is driven by real systems rather than a feature list. Open with authentic deployments and let differences surface from the work those systems are doing, then return to context at the end to surface the ethical dimension—the energy and environmental cost of large-scale GPU compute.

Spotlights taken from the unit overview: A1 Computer Fundamentals.

2. Lesson Sequence (One-Hour Block)

Part 1: Do Now (Retrieval) — ~10%

Activity: Three short recall items from earlier taught content, tagged to source:

Question	Source
1. What are the layers of cache memory, and why is cache small and fast rather than large and slow?	[A1.1.4]

Question	Source
2. What does the CPU do in the machine instruction cycle, and why must it handle varied, branching work?	[A1.1.1]
3. What is parallel processing, and what kind of work suits it?	[A1.1.2]

Part 2: Main Sequence — ~60%

Phase A: Anchor — Three Real Systems

Present three real systems briefly:

System	Work Being Done	Work Shape
 Games Console	Rendering frames, game logic, physics	Mixed: varied decisions + parallel pixel processing
 AI Training Cluster	Training neural networks on massive datasets	Same operation repeated over huge amounts of data
 Scientific Simulation	Weather modelling, molecular dynamics	Same calculation over millions of independent grid cells

Task: Students record their sorting before any theory is given—classifying each workload as:

- Same operation repeated over huge amounts of data → GPU-suited
- Varied decisions taken one after another → CPU-suited

Phase B: Teach Through the Anchors — Four Comparison Dimensions

Work through the four comparison dimensions, giving the **reason** for each and mapping it back to the anchored systems:

Dimension	CPU	GPU	Reason
Core Architecture	Few complex cores	Many simple cores	CPU cores handle varied, branching instructions; GPU cores repeat the same simple operation
Processing Power	Per-core speed and low latency	Combined parallel throughput	CPU minimises delay per task; GPU maximises total work completed
Memory Access	Layered cache (L1, L2, L3)	Wide, high-bandwidth memory	CPU needs fast access to small data; GPU needs massive data flow
Power Efficiency	Per-core efficiency	Per-workload efficiency	CPU efficient per operation; GPU efficient per unit of work completed

Teaching Note: Keep every point at the level of **why**, in line with the Explain command term.

Phase C: Collaboration — How CPU and GPU Work Together

Teach the three aspects of collaboration:

Aspect	Description
Task Division	CPU handles control flow, branching, and complex logic; GPU handles parallel, repetitive calculations
Data Sharing	Data transferred over PCIe (Peripheral Component Interconnect Express) bus
Execution Coordination	CPU orchestrates the overall program; GPU executes specific parallel tasks when instructed

Visible in the same systems: The partnership between CPU and GPU in games consoles, AI clusters, and scientific simulations.

Phase D: Return with the Ethical Dimension

- Come back to the **AI training cluster**
- Connect the power-efficiency dimension to the **energy and environmental cost** of running GPU compute at scale
- This is the contexts return and the hook for the research strand

Part 3: Deliberate Practice — ~20%

Activity (Independent or Pairs):

Students write a causal explanation using the eBook prompt:

| "Explain why a GPU renders a game frame faster than a CPU could, even though each individual CPU core is faster."

Hold responses at reasons and causes—a correct answer names the workload shape, not a "winning" processor.

Sentence Frame:

| "A GPU renders a game frame faster because rendering involves [workload shape], which matches the GPU's [architectural feature], whereas the CPU is better suited to [different workload type]."

Part 4: Exit Check — ~10%

Two short items the teacher can read in minutes:

Question	Purpose
(a) Give one reason the GPU's cores can be simple and numerous while the CPU's must be complex and few	Assesses understanding of core architecture differences
(b) Name one task in your researched system that must run on the CPU, and say why	Closes the loop on the Research ATL

Assessment: Review responses to ensure students are explaining reasons and causes, not just describing features.

3. Assessment & Exemplar Responses

Exemplar Response — Deliberate Practice

| "A GPU renders a game frame faster than a CPU because rendering involves performing the same operations on millions of individual pixels simultaneously. The GPU has thousands of simple cores designed for exactly this type of parallel work, so all those pixel calculations happen at the same time. The CPU, with its few complex cores, would have to process each pixel one after another, which takes much longer even though each CPU core is individually faster."

Research Task — Criteria

Criteria	Met / Not Met
Identified a credible source	☐
Extracted how work is split between CPU and GPU	☐
Recorded where the source came from	☐
Explained why the workload shape determines the split	☐

4. Differentiation & Extension

Support (Scaffolding That Can Be Removed)

Support Strategy	Description
Sentence Frame	Provide for the deliberate practice task
Partially Completed Comparison Grid	Dimension names filled in, reasons left blank
Curated Source for Research	Provide a specific source rather than open search; withdraw once students can locate their own

Challenge (Deeper or Wider)

Challenge Strategy	Description
PCIe Bottleneck Investigation	Investigate why transferring data over PCIe is a bottleneck and how batching work into large chunks reduces the cost of shuttling data back and forth
Independent Research	Research a specific system's CPU-GPU split without a curated source and explain the design choices behind it

5. Engagement Activities (Supplementary)

If time allows, these can replace or supplement the main sequence:

Activity	Description
"Divide and Conquer" Analogy	Present a complex task (e.g., building a house) and discuss how dividing among specialists leads to better results
"Guess the Processor" Activity	Show images/clips of different computing tasks; students guess which processor is better suited
"CPU vs. GPU Face-Off"	Divide into two teams; present tasks and have each team argue why their processor is better suited
"Explain in Simple Words"	Teams explain strengths of their assigned processor as if to a non-specialist

Activity	Description
"Task Allocation Challenge"	Students analyse tasks and determine workload division between CPU and GPU for optimal performance
Case Study Discussion	Real-world case study of CPU-GPU collaboration in gaming, AI, or scientific research

6. Learning Objectives (Summary)

Knowledge and Understanding

- Differentiate between the design philosophies and primary uses of CPUs and GPUs
- Compare and contrast core architecture, memory access, and power efficiency
- Explain how CPUs and GPUs collaborate to achieve optimal performance

Skills

- Analyse the strengths and weaknesses of CPUs and GPUs for different task types
- Explain task division, data sharing, and execution coordination between CPUs and GPUs
- Evaluate the suitability of CPUs and GPUs for various real-world applications

ATL Skills

Skill	Description
Communication	Effectively communicate understanding through discussions, presentations, and written explanations
Collaboration	Work collaboratively to analyse diagrams, solve problems, and complete activities
Critical Thinking	Analyse trade-offs between architectures; evaluate suitability for different tasks
Information Literacy	Effectively use online resources and research to deepen understanding

7. Resources

- **Presentation** (Slide Deck)
- **Worksheet:** CPU vs. GPU Comparison Table
- **Worksheet Answers**
- **InThinking eBook** (A 1.1.3) with quiz

Author's Reflection

"I have rebuilt this lesson around contexts because the differences between the two processors only make sense once you can see where each one earns its place. It starts with real systems—a games console, an AI training cluster, a scientific model—and lets the four comparison dimensions fall out of the work those systems are doing rather than arrive as a table to memorise.

The command term is Explain, so I have held the lesson at reasons and causes throughout: students say why the GPU's simple cores can be replicated by the thousand, not which processor is 'better', which is the trap the markband punishes. The one trade-off I am conscious of is that A1.1.2 already leads on

contexts, so I have kept these anchors on the CPU-GPU partnership and the energy cost of parallel compute to stop the two lessons treading the same ground.

For the research element, students investigate one real deployment and report how the work is split."